

Day5. REST API 구현

1. 수업내용 정리

1.1. Spring Security

1.1.1. Spring Security 필터 체인 - HTTP 요청 처리 순서

1. HTTP 요청
2. SecurityFilter Chain
3. JWT Filter
4. Authentication Manager
5. Controller

1.1.2. 인증(Authentication) vs 인가(Authorization)

인증

- 너가 누구인지만 판별
- JWT 토큰 검증

인가

- 권한(Role) 체크

1.2. MSA

1.2.1. MSA가 필요한 이유

- 독립적으로 서비스 배포
- 기술 스택 자유
- 장애 격리
- 수평 확장 용이

그러나 단점도 존재함

- 네트워크의 복잡도가 증가
- 분산 트랜잭션이 어려움
- 운영 비용이 높음

그러면 서비스끼리의 연결은? 자동으로 라우팅되어서 찾아가게. (kafka)

2. 코드 분석

이번 실습코드는 기존 Day4 - 게시글 작성에서 댓글과 좋아요 버튼이 추가된 코드.

파일 트리구조는 다음과 같음:

```
loginauth
├── auth
│   ├── domain
│   ├── dto
│   ├── exception
│   ├── repository
│   ├── service
│   └── web
└── post
    ├── domain
    ├── dto
    ├── exception
    ├── repository
    ├── service
    └── web
```

```
├── comment (추가)
│   ├── domain
│   ├── dto
│   ├── exception
│   ├── repository
│   ├── service
│   └── web
├── like (추가)
│   ├── domain
│   ├── dto
│   ├── exception
│   ├── repository
│   ├── service
│   └── web
└── global
    ├── config
    ├── exception
    ├── security
    └── web
```

2.1. comment

domain

```
package loginauth.comment.domain;

import jakarta.persistence.*;
import loginauth.post.domain.Post;

import java.time.LocalDateTime;

@Entity
@Table(name = "comments")
public class Comment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "post_id", nullable = false)
    private Post post;

    @Column(nullable = false, length = 50)
    private String writer;

    @Lob
    @Column(nullable = false, columnDefinition = "TEXT")
    private String content;

    @Column(name = "created_at", nullable = false)
    private LocalDateTime createdAt;

    @Column(name = "updated_at", nullable = false)
    private LocalDateTime updatedAt;

    protected Comment() {
    }

    public Comment(Post post, String writer, String content) {
        this.post = post;
        this.writer = writer;
        this.content = content;
        this.createdAt = LocalDateTime.now();
        this.updatedAt = LocalDateTime.now();
    }
}
```

```

public void update(String content) {
    this.content = content;
    this.updatedAt = LocalDateTime.now();
}

public Long getId() { return id; }
public Post getPost() { return post; }
public String getWriter() { return writer; }
public String getContent() { return content; }
public LocalDateTime getCreatedAt() { return createdAt; }
public LocalDateTime getUpdatedAt() { return updatedAt; }
}

```

- Comment는 Post에 소속되는 하위 리소스
- URL도 `/api/posts/{postId}/comments` 형태로 게시물 아래에 댓글이 존재한다는 구조를 표현
- `@ManyToOne`은 여러 댓글이 하나의 게시물에 연결될 수 있다는 의미 / `FetchType.LAZY`는 댓글을 조회할 때 게시물 정보를 즉시 가져오지 않고, 필요할 때 가져오도록 지연 로딩 설정.
- `@Lob`은 대용량 데이터(긴 텍스트, 이진 데이터) 다룰 때 사용.
- `@JoinColumn`은 엔티티 간에 연관관계(1:N, N:1, 1:1 등)를 맺을 때, DB 테이블의 외래 키 컬럼을 지정하는 역할.

web/controller

```

package loginauth.comment.web;

import jakarta.validation.Valid;
import loginauth.comment.dto.CommentCreateRequest;
import loginauth.comment.dto.CommentResponse;
import loginauth.comment.dto.CommentUpdateRequest;
import loginauth.comment.service.CommentService;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import java.net.URI;
import java.util.List;

@RestController
@RequestMapping("/api/posts/{postId}/comments")
public class CommentController {

    private final CommentService commentService;

    public CommentController(CommentService commentService) {
        this.commentService = commentService;
    }

    @GetMapping
    public List<CommentResponse> findAll(@PathVariable Long postId) {
        return commentService.findAll(postId);
    }

    @PostMapping
    public ResponseEntity<CommentResponse> create(
        @PathVariable Long postId,
        @Valid @RequestBody CommentCreateRequest request,
        Authentication authentication
    ) {
        CommentResponse response = commentService.create(
            postId,
            request,
            authentication.getName()
        );

        return ResponseEntity
            .created(

```

```

        URI.create(
            "/api/posts/" + postId
            + "/comments/" + response.id()
        )
    )
    .body(response);
}

@PutMapping("/{commentId}")
public CommentResponse update(
    @PathVariable Long postId,
    @PathVariable Long commentId,
    @Valid @RequestBody CommentUpdateRequest request,
    Authentication authentication
) {
    return commentService.update(
        postId,
        commentId,
        request,
        authentication.getName()
    );
}

@DeleteMapping("/{commentId}")
public ResponseEntity<Void> delete(
    @PathVariable Long postId,
    @PathVariable Long commentId,
    Authentication authentication
) {
    commentService.delete(
        postId,
        commentId,
        authentication.getName()
    );

    return ResponseEntity.noContent().build();
}
}

```

- **POST**는 새로운 리소스를 생성할 때 사용.
- **PUT**은 기존 리소스를 수정할 때 사용.
- **DELETE**는 기존 리소스를 삭제할 때 사용.
- POST가 다른 REST API와 다른 점은 **비 멍등성**임
- **멍등성**은 어떤 작업을 여러 번 실행하여도 결과와 서버의 상태가 같은 상태.

2.2. like

domain

```

package loginauth.like.domain;

import jakarta.persistence.*;
import loginauth.post.domain.Post;

import java.time.LocalDateTime;

@Entity
@Table(
    name = "post_likes",
    uniqueConstraints = @UniqueConstraint(
        name = "uk_post_likes_post_username",
        columnNames = {"post_id", "username"}
    )
)
public class PostLike {

```

```

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "post_id", nullable = false)
private Post post;

@Column(nullable = false, length = 50)
private String username;

@Column(name = "created_at", nullable = false)
private LocalDateTime createdAt;

protected PostLike() {
}

public PostLike(Post post, String username) {
    this.post = post;
    this.username = username;
    this.createdAt = LocalDateTime.now();
}

public Long getId() { return id; }
public Post getPost() { return post; }
public String getUsername() { return username; }
public LocalDateTime getCreatedAt() { return createdAt; }
}

```

- 좋아요는 게시글과 사용자 사이의 관계를 저장하는 Entity
- 한 사용자가 같은 게시글에 좋아요를 여러 번 누르면 안되므로 `@UniqueConstraint`를 통해 `post_id + username` 조합으로 제약조건을 걸어둠
- 좋아요는 별도의 id를 URL에 노출하지 않고, 현재 로그인 사용자와 `postId` 조합으로 대상을 찾음.

web/controller

```

package loginauth.like.web;

import loginauth.like.dto.PostLikeResponse;
import loginauth.like.service.PostLikeService;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/posts/{postId}/likes")
public class PostLikeController {

    private final PostLikeService postLikeService;

    public PostLikeController(PostLikeService postLikeService) {
        this.postLikeService = postLikeService;
    }

    @GetMapping
    public PostLikeResponse status(
        @PathVariable Long postId,
        Authentication authentication
    ) {
        return postLikeService.status(
            postId,
            authentication.getName()
        );
    }

    @PostMapping

```

```

public PostLikeResponse like(
    @PathVariable Long postId,
    Authentication authentication
) {
    return postLikeService.like(
        postId,
        authentication.getName()
    );
}

@DeleteMapping
public PostLikeResponse unlike(
    @PathVariable Long postId,
    Authentication authentication
) {
    return postLikeService.unlike(
        postId,
        authentication.getName()
    );
}
}

```

- 어노테이션에 경로 변수가 있을 경우와 없을 경우의 차이점.
- `@GetMapping`은 현재 경로 그대로 요청을 처리함.
- `@GetMapping("/{postId}")`는 하위 경로에 id가 붙은 요청을 처리함.
- 즉, 경로 변수가 있으면 특정 리소스 하나를 지정하고, 없으면 보통 목록이나 상태를 조회함.

2.3. Docker

```

# 1단계: Gradle로 Spring Boot 실행 JAR 빌드
FROM eclipse-temurin:21-jdk AS builder

# 내부적으로 /workspace 폴더를 만들고, 그 안에서 빌드 진행
WORKDIR /workspace

# Gradle 빌드 캐시를 도커 볼륨으로 활용하기 위해, 캐시를 저장할 폴더를 미리 만들어둠(COPY 앞 뒤: 앞을 뒤로 복사)
COPY gradlew ./
COPY gradle ./gradle
COPY settings.gradle ./
COPY build.gradle ./

# 실행 권한 부여 - chmod는 파일이나 디렉토리의 권한을 변경하는 명령어
RUN chmod +x ./gradlew

COPY src ./src

# Gradle 빌드 실행 -> 결과: /workspace/build/libs/*.jar
RUN ./gradlew clean bootJar \
    --no-daemon \
    --no-configuration-cache

# 2단계: 실행 이미지
FROM eclipse-temurin:21-jre

WORKDIR /app

RUN groupadd --system spring \
    && useradd --system \
        --gid spring \
        --create-home \
        spring

COPY --from=builder \
    /workspace/build/libs/*.jar \

```

```
/app/application.jar
```

```
RUN chown spring:spring /app/application.jar
```

```
USER spring:spring
```

```
EXPOSE 9999
```

```
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75.0", "-Djava.security.egd=file:/dev/./urandom", "-jar", "/app/application.jar"]
```

- 해당 파일은 크게 2페이지로 나뉨
- 1 페이지는 Gradle을 이용해 Spring Boot jar 파일을 빌드
- 2 페이지는 빌드된 jar 파일만 복사해서 실행
- `eclipse-temurin:21-jdk`는 빌드에필요한 이미지.
- `eclipse-temurin:21-jre`는 실행에 필요한 이미지.
- `EXPOSE 9999`는 컨테이너가 9999 포트를 사용한다는 문서적 표시
- `ENTRYPOINT`는 컨테이너가 시작될 때 실행할 명령어.

2.3.1. Docker vs Docker-Compose vs K8s

구분	Docker	Docker-Compose	K8s
목적	컨테이너 하나 실행	여러 컨테이너를 함께 실행	컨테이너를 운영 환경에서 자동 관리
설정 파일	Dockerfile	docker-compose.yml	YAML 매니페스트
사용 예시	Spring Boot 앱 하나 실행	앱 + DB + Redis 함께 실행	여러 서버에 앱 배포/확장
장점	단순하고 빠름	로컬 개발 환경 구성에 편함	자동 복구, 확장, 무중단 배포 가능
단점	여러 서비스 관리가 불편	운영 환경 관리에는 한계	학습 난이도와 운영 복잡도가 높음